TEAM INITIATORS · DISTRICT IQ REVIEW · 18 OCT 2026, 4:45 PM

Website demo script

For [your name] · about 20 minutes: Collector, officer, citizen, click by click (11-minute version: the parts marked short)

About 20 minutes. The script follows the screen in order (Collector, then officer, then citizen, then back to the Collector), so you never jump back. Speak it like you are showing a friend something you built and are proud of: short sentences, look up from the page, let the screen do half the work. Yellow lines are what to click. Parts marked "short" make an 11-minute version.

1 · Opening `0:00 – 1:00 · short`

DO **Collector console is open on Overview. Hands off the mouse.**

SAY

Thank you, [friend's name].

So, what you're looking at is the Collector's screen. This is what the Collector of Chennai would open at eight in the morning.

And I want to start with a simple question, because everything we built comes from it. Overnight, thousands of things have happened in this district. Complaints, police calls, flood readings, news in two languages. **Which of these need the Collector today? And why?**

Most dashboards show you more data. We wanted to do the opposite: show **less, but the right things**, and always explain why. Let me show you how.

TECH / ALGORITHM

Next.js 14 + React + TypeScript console; each hourly build loaded from AWS S3 into in-memory SQLite (node:sqlite), refreshed automatically.
Every number traceable to its source record and source file (SHA-256 snapshot on every row).

2 · The data, and the first problem we hit `1:00 – 3:00 · short`

DO **Click the "Collected …" chip in the top bar. Data sources opens.**

SAY

Everything starts here. This little chip tells the Collector when the data was last collected and whether every source is up to date. Right now, all eight are current, and this refreshes **every hour, on its own**.

These are our sources: IMD weather and its district warnings, the Chennai Flood Monitor's river levels, CPCB air quality, Chennai Metro Water's lake levels, market prices, and news from six Chennai newspapers in English and Tamil.

Now, the first real problem we hit: **government data is not built to be collected.** CPCB has no open feed at all. And two of these sites simply block cloud servers, so our hourly job kept failing.

We didn't want to cut corners there. So for CPCB, we read the public map the same way a citizen sees it in a browser. For the blocked sites, our own machine collects them and passes them to the cloud job. To the Collector, none of that is visible. The data is just there, every hour.

And one rule we never broke: if a site has a captcha or a login, **we stop.** We never bypass it. For a government platform, we felt how you collect data matters as much as what you collect.

SAY

One more thing, so you know exactly what you're seeing. Grievances, police, PWD and hospitals don't publish open data, so we built realistic simulators for them. The challenge was that they didn't agree with each other: each one had its own idea of when it rained. So we gave them **one shared world**, with one rain calendar and Chennai's real wards. Now when it rains, complaints, police calls and drain work all rise together, just like real life. Every simulated record is marked, and plugging in a real department later is one small connector.

DO **Close the dialog.**

TECH / ALGORITHM

Python collectors: requests (IMD, Flood Monitor APIs), Playwright headless browser (CPCB), feedparser + trafilatura (news).

Hourly on GitHub Actions triggered by cron-job.org; PC Task Scheduler for CPCB and CFM; result stored in AWS S3.

3 · What's happening: the Overview `3:00 – 4:30 · short`

DO **Point at the four tiles. Switch the period from Daily to Weekly.**

SAY

So this is the morning view. Severe events, open complaints, ongoing incidents, and what's been resolved, each compared with the previous period, so the Collector instantly sees whether things are getting better or worse.

The Collector picks the time frame: today, this week, this month or this quarter. And everything on the page follows that choice.

DO **Click a zone on the map. Watch the page change. Then clear it.**

SAY

Now watch this. When I click a zone, **the whole console becomes that zone**. The numbers, the lists, the news, the briefing. A zonal review meeting becomes one click.

Here's a small detail that turned out to matter a lot. A lot of news just says "in Chennai", with no street or area. If we pinned all of those, they would pile up in one spot and draw a huge hotspot that doesn't exist. So if we don't know exactly where something happened, **we don't put it on the map**. We list it separately instead. The map should never mislead the Collector.

DO **In Latest news, click the "Next door" tab.**

SAY

And this tab is one of my favourites. Chennai doesn't flood only because of Chennai. If a lake upstream in Tiruvallur or Chengalpattu releases water today, it reaches Chennai tomorrow. So **Next door** watches the news from the three neighbouring districts and flags what could reach us. It's early warning from beyond the district boundary.

TECH / ALGORITHM

Leaflet map on GCC ward and zone boundaries (GeoJSON); KPI tiles computed by SQL for the chosen period and zone; ECharts.

Point-in-polygon placement; items without an exact place are not pinned; Next door = news from the three neighbouring districts.

4 · One incident, from seven reports `4:30 – 7:30 · short`

DO **Open the severe incident you chose before the meeting (one with several sources and news).**

SAY

Let me open one incident. This is the part of the project we're proudest of, and honestly the part that took the longest.

In the real world, one problem doesn't arrive as one report. A waterlogged street comes in as three citizen complaints, a police report, a PWD work order, and two news articles, one in Tamil and one in English. To a normal system, that's seven separate things. **Here, it's one incident.** One timeline, and the evidence from every source right here.

SAY

Getting this right was hard, and I'll tell you about three problems we had to solve.

The first: Tamil and English. A Tamil article and an English article about the same accident share almost no words. Our first approach simply couldn't tell "the same accident" apart from "two different accidents on the same day". So we rebuilt it to compare **meaning**, not words, using a multilingual AI model that understands both languages. A second model then reads the two articles side by side, the way a person would. Today, about **2,800 Tamil and English reports** are matched this way.

The second: chains. If report A looks like B, and B looks a bit like C, a naive system pulls all three together, even when A and C are completely unrelated. We saw that happen, so we stopped it. Every match has to stand on its own.

The third, and this one taught us the most: coincidences. When we checked real news against our department records by hand, 141 pairs of them, we found the system sometimes joined two events just because they happened at the same place on the same day. A road-rage report and a murder story, for example. So we added a simple rule: they must also be the same kind of incident. **Wrong joins dropped from 32 to 9.** That's how we've worked the whole way through: test on real data, find what's wrong, fix it.

The result: when this system says two reports are the same incident, it's right **97 times out of 100.** And when it isn't sure, it doesn't guess. It puts the pair on a review list for a person.

TECH / ALGORITHM

Duplicates: blocking + SimHash + union-find. Linking: KD-tree + logistic regression on 9 features + multilingual-e5 embeddings.
Tamil–English stories: bge-m3 embeddings + bge-reranker-v2-m3 cross-encoder + trained classifier + LLM check.

---

5 · Why this one is at the top `7:30 – 9:00 · short`

DO **Point at the severity, the priority and the list of reasons in the pop-up.**

SAY

Now, the second question: why does this need the Collector first?

Most systems give you a score and expect you to trust it. **We give you the reasons.** You can read them right here: severe, past its deadline, not yet checked on the ground, vulnerable people nearby, linked to rain. Every point is explained. Nothing is a black box.

We had to think carefully about one trap here. One busy street can collect thirty complaints about the same pothole, while a single report of a live electric wire is far more dangerous. If every complaint simply added points, the pothole would win. That's the wrong order. So complaints count with diminishing weight: more complaints still raise priority, but they **can never outweigh real danger**.

And here's what makes it smarter over time: **it learns from the Collector.** Every time the Collector verifies, reopens or escalates something, that decision is recorded. After thirty decisions, the system adjusts its own weighting to match the Collector's judgement. And it only switches to the learned version if that version actually ranks better. So it adapts, but it can't drift off course.

TECH / ALGORITHM

Severity: rule-based score 0–100. Priority: weighted signals, log scale for sources and complaints.
Learning: logistic regression with a Gaussian (Bayesian) prior, adopted only if held-out ranking (AUC) improves.

---

6 · From insight to action `9:00 – 10:30 · short`

DO **Scroll to the AI analysis, then "Reports differ" if present. Then click "Take action". Show the draft. Do not send.**

SAY

Down here, the AI has read every report behind this incident and summarised it. And here's the important part: **every number it writes is checked against the original reports** before you see it.

And this, **Reports differ**, came from something we noticed in the news. Different papers often report different death tolls, or a different age, or even a different place. If a Collector quotes the wrong figure in a press meet, that's a real problem. So we flag every disagreement, side by side.

Now, seeing a problem isn't enough. The Collector has to act on it. So I click **Take action**, and an AI agent drafts the instruction to the right department officer, with the facts and the deadline already in it. But **nothing is sent until the Collector approves it.** AI prepares, the human decides.

Then the officer does the work and sends back a completion report with photos from the site. It comes straight back here, into **My Tasks**, and the Collector either verifies it or sends it back. Every step is recorded. That's real accountability between the Collector and every department, in one loop. Let me show you that loop from the other side.

TECH / ALGORITHM

AI analysis and drafts: Amazon Bedrock (Nova Lite), Gemini and Groq as fallbacks, every figure checked by a number verifier.
Reports differ: LLM side-by-side comparison with figure check; email via Nodemailer; decisions and audit log in DynamoDB.

---

7 · The department officer's side `10:30 – 12:30 · short`

DO **Switch to the second browser tab, already signed in as a department officer (for example Electrical or Storm Water Drain). If you sign in live, do it off the shared screen.**

SAY

So that's the Collector's side. But a Collector can't fix a drain. The departments do the work. So let me switch to the other side of the loop: a **department officer**.

And notice: I'm still signed in as the Collector in the other tab. Each tab keeps its own login, so the Collector, an officer and a citizen can all be open side by side. We actually had to fix that: at first, logging out in one tab logged you out everywhere, which made exactly this kind of demo impossible.

This officer sees **only their own department**. Their severe events, their open incidents, what is past its deadline, what is **due in the next 24 hours**, and which zone has the most open work. It's the same system as the Collector's, but focused on one job.

DO **Point at "What needs you now", then open My Work and walk through its tabs: New → In action → Sent to Collector → Verified.**

SAY

Here is the officer's work queue, and it follows a simple journey: **New, In action, Sent to Collector, Verified.**

A new grievance arrives, and the officer approves it, which puts it in action. When the work is done, they send a completion report. And here we made a deliberate choice: **you can't just tick a box.** The report must have remarks describing the work, and **at least one photo of the completed work**. Proof, not a promise.

Then a second challenge: if every completed job went to the Collector, the Collector would drown in approvals. So only the **severe** ones go to the Collector to verify. Everything else, the department closes itself. The Collector's time goes where it matters most.

DO **Point at "Collector Feedback" (work returned by the Collector), and open one grievance to show the instruction "From the Collector" if there is one.**

SAY

And if the Collector isn't satisfied, the work comes back here, under **Collector Feedback**, to be redone. The instruction the Collector approved earlier shows up right inside the grievance, "From the Collector", with the facts and the deadline.

Officers also get Ask District IQ, but locked to their department. An electrical officer can ask anything about streetlights, and nothing about the police. Even if they try, the system simply won't show another department's data.

TECH / ALGORITHM

Next.js + React + TypeScript console; server-side role guard and department lock; per-tab JWT sessions (jose).
Completion report with photo upload; saved to DynamoDB through API Gateway + Lambda.

---

8 · The citizen's side `12:30 – 14:30 · short`

DO **Switch to the third tab, signed in as a citizen. Show the home page, then click "File a complaint".**

SAY

And finally, where it all starts: **the citizen.**

Citizens can sign up on their own. Officer and Collector accounts are issued, never self-registered, because a government system has to know exactly who is on the inside.

The first challenge we thought about here: **a citizen doesn't know their ward number, or which department handles a fallen streetlight.** They shouldn't have to. So filing is four simple steps: your details, the location, the type of problem, and the details.

DO **Step 2: drop a pin on the map. Show the ward and zone filling in by themselves. Step 3: pick a complaint type. Show the department filling in. Step 4: add a short description and a photo.**

SAY

I just point to the problem on the map. [Drop the pin.] The system works out the **ward and zone from the ward boundaries itself.** I pick what kind of problem it is, and the **right department is chosen automatically**. If the citizen picks "Other" and just describes it in their own words, the AI reads it and routes it to the right department, and an officer confirms.

A photo or a short video can be attached, and citizens can even file **anonymously** if they're afraid to give their name.

DO **Click Submit. Show the complaint number.**

SAY

Submitted. Here's the complaint number.

And a small story here. On a slow connection, people double-click, or press submit again, and we found that the same complaint was being registered twice. So now a second press simply waits for the first and gets the **same complaint number back**. No duplicates, and the citizen isn't confused.

DO **Go to Track status and open a complaint that has been resolved (choose it before the meeting). Show its journey and "Photos of the work done".**

SAY

Now this is my favourite part of the citizen side. The citizen can **track every complaint** and see its whole journey, step by step. And when the work is done, they see the officer's remarks and the **photos of the work done.**

So the same photo the officer uploaded as proof for the Collector also reaches the citizen. One loop, from the citizen, to the department, to the Collector, and back to the citizen. **Everyone sees the same truth.**

Rehearse this part once before 4:45: sign in to all three tabs (Collector, one officer, one citizen) in the same browser. Pick a resolved complaint with photos for the citizen's Track page. Decide whether to file a new complaint live; if you do, use a clearly test description. Don't type passwords on the shared screen.

TECH / ALGORITHM

Leaflet map + point-in-polygon on GCC ward boundaries (GeoJSON); department from the complaint type, "Other" routed by AI or a keyword classifier.

bcrypt passwords, encrypted ID numbers, a one-at-a-time lock that stops double submissions.

---

9 · The morning briefing `14:30 – 15:30`

DO **Go to the Briefing page. Scroll slowly. Switch to Tamil, then back.**

SAY

Every morning, the Collector also gets a briefing. If they read nothing else, they read this.

The key points at the top are deaths, severe incidents and IMD warnings, picked by fixed rules, so **nothing critical depends on an AI's mood.** Then a short brief for each department, the stories still developing, and something we think is really valuable: things that are **in the news but no department has a record of yet**. Those are exactly the things that slip through the cracks.

Every number in this briefing is checked against the data. Today there are zero numbers we couldn't verify. And [click] the whole thing is available in Tamil.

TECH / ALGORITHM

Briefing agent: key points by fixed rules, text by AI, every number checked by a numeric verifier (0 unverified).

Bilingual English/Tamil; PDF export with jsPDF.

---

10 · Warnings people can trust `15:30 – 16:45`

DO **Trends page → Unusual rises. Then Recurring hotspots.**

SAY

Here's a challenge every alert system faces. If it cries wolf too often, people stop reading it. When we started, a simple "is today higher than usual?" rule gave us more than twenty alerts a week, and most of them were noise.

The big reason was **rain**. Of course waterlogging complaints jump when it rains. That's not news. So now every problem type, in every zone, learns its own normal level, including how much it normally rises on rain days and on particular weekdays. On top of that, we run a statistical correction, because when you check four thousand things a day, some will look unusual just by chance.

Today, the old rule would have raised fifteen alerts. Ours raised **one**. So when this page says something is rising, it's worth the Collector's time.

And here, **recurring hotspots**: places where the same problem keeps coming back. Today we found about 330. These are the places where one permanent fix, a new drain or a new streetlight, saves a hundred future complaints.

DO **Optional, 15 seconds: click Environment & markets.**

SAY

And for completeness: rainfall, air quality, reservoirs, lake levels and vegetable prices, all live, on one page.

TECH / ALGORITHM

Poisson baselines with weekday and rain factors; spike test + CUSUM; Benjamini-Hochberg false-discovery control at 5%.
Hotspots: DBSCAN (haversine distance, 5 incidents within 250 m) + Getis-Ord Gi* on wards.

---

11 · Just ask `16:45 – 18:30 · short`

DO **Click Ask. Type "theft cases by zone this month". Then "show as a chart". Pin it. Then ask one question in Tamil.**

SAY

Now, not every Collector wants to click through pages. Sometimes you just want to ask. So we built **Ask District IQ**.

[Type the question.] There's the answer, by zone. [Ask for a chart.] Now as a chart. [Pin it.] And it's pinned to the dashboard.

Here's the challenge we had to solve. AI chatbots are great with words, but they **make up numbers.** In government, a made-up number is not acceptable. So in ours, the AI is never allowed to produce a number. It only understands the question. The actual figures come straight from the data, and then **every number in the answer is checked** before you see it.

When we first tested it on 75 questions with known answers, it got **37 right**. We went through every failure, one by one. One example: asked about incidents "in ward 120", it searched for the text "ward 120" and said zero. Now it reads that ward's own records. Today it gets **all 75 right, even with the AI switched off.** So if every AI service went down tomorrow, the Collector would still get correct answers.

[Ask in Tamil.] And of course, it works in Tamil too. Department officers get the same assistant, but it only ever shows them their own department.

TECH / ALGORITHM

Router → query planner → compiler → read-only SQL on in-memory SQLite (node:sqlite); number verifier on every answer.
Search: LanceDB hybrid (multilingual-e5-base vectors via Transformers.js + BM25 keywords, reciprocal-rank fusion).

---

12 · Add any new source with a link `18:30 – 19:30`

DO **Data Studio → Add data → paste the link you prepared. Let the preview appear.**

SAY

Last feature. A district never stays the same. There will always be a new website, a new feed, a new department report. So the Collector shouldn't need a developer to add one.

I just paste a link. [Paste.] The system checks it's allowed to read the site, works out what kind of source it is, maps the fields, and shows me a **preview before anything is saved**.

And here's a choice we're quite proud of. AI is used **only once**, at this setup step. After that, the source runs every hour on its own, with **no AI cost at all**. So adding twenty new sources doesn't mean twenty new AI bills.

TECH / ALGORITHM

Onboarding agent: robots.txt + SSRF guard, detects RSS / API / page / login, Gemini maps the fields, preview before save.
Scheduled runs use the saved mapping with zero AI calls; uploaded datasets profiled with SheetJS.

---

13 · Close `19:30 – 20:30 · short`

DO **Go back to the Overview. Hands off the mouse. Look at the judges.**

SAY

So let me bring it back to that eight o'clock question.

With District IQ, the Collector sees **one incident instead of seven reports.** They're warned early, from upstream districts and from rises that are actually real. They know why something is at the top of the list. They can act on it in two clicks, and check that it was actually done. The officer gets a clear queue and has to show proof. And the citizen who complained sees the photo of the finished work. All of it works in Tamil as well as English.

It runs entirely on the AWS services we were given, refreshes itself every hour, and keeps working even when the AI doesn't.

We know where we go next. Our news filter is our weakest point today, and labelling more real news will improve it. And the biggest step is connecting one real department system. We built it so that's one connector, not a rebuild.

We've put a lot of ourselves into this, and we really believe it can change how a Collector's day starts. Thank you. We'd love to take your questions.

TECH / ALGORITHM

Front end: Next.js, React, TypeScript, Leaflet, ECharts. Pipeline: Python, pandas, scikit-learn, sentence-transformers.

AWS: S3, DynamoDB, Lambda, API Gateway, Bedrock (ap-south-1, Mumbai).

Challenges we solved (learn these: judges love them)

Challenge	How we solved it	Result
CPCB has no open feed	Read the public map like a citizen's browser	Live AQI every hour
Two government sites block cloud servers	Our own machine collects them and hands them to the cloud job	All 8 feeds current
The cloud scheduler skipped runs	An external hourly trigger, and a check that skips an hour already built	Reliable hourly refresh
Simulators disagreed on rain days	One shared world: one rain calendar, real wards	Sources rise together on rain days
Tamil and English share no words	Match by meaning, then a second model reads each pair	About 2,800 cross-language matches
Chains pulling in unrelated reports	Every match must stand on its own	No accidental mega-incidents
Joins by coincidence of place and time	Checked 141 real pairs by hand; added a same-category rule	Wrong joins 32 → 9; 97% precision
"In Chennai" news making fake hotspots	Don't pin what we can't place	A map that never misleads
Thirty duplicate complaints outranking real danger	Complaints count with diminishing weight	Danger stays on top
Too many false alerts	Rain- and weekday-aware normal levels, plus a false-alarm correction	1 alert instead of 15 today
Newspapers disagree on figures	"Reports differ" flags every conflict	The Collector never quotes a wrong figure

Chatbots invent numbers	AI never produces numbers; every figure is checked	37/75 → 75/75, even with AI off
AI keys expire, services go down	Backup AI providers, and answers from the data when all are down	Never a blank screen
Logging out in one tab logged out every tab	Each tab keeps its own login	Collector, officer and citizen side by side
"Done" with no proof	Completion report needs remarks and at least one photo	Proof, not a promise
Collector flooded with approvals	Only severe work goes to the Collector; departments close the rest	Collector's time goes where it matters
Citizens don't know their ward or department	Map pin fills ward and zone; type picks the department; AI routes "Other"	Filing in four simple steps
Double-click registered a complaint twice	A second press waits and returns the same complaint number	No duplicate complaints
Citizens never see what was done	Officer's remarks and work photos shown on the Track page	Everyone sees the same truth
New sources need a developer	Paste a link; AI maps it once, then it runs free	New source in minutes, no ongoing AI cost

How to deliver it: say each challenge as a short story: "here's a problem we hit… so we… and now…". Pause after the bold lines. Let the screen change before you explain it. If something fails live, smile and say: "this refreshes hourly; let me show you the same thing from here." If the AI is down, say: "this is exactly the fallback I mentioned. The answer still comes from the data."